Binary Search Tree Basics

Design and Analysis of Algorithms

Brian C. Dean

School of Computing

Clemson University

Set / Map Data Structures

- Set / Dictionary: maintain dynamic collection of elements
  - Insert (k) : insert key k
  - Remove (k) : remove key k
  - Find (k) : is key k present?
- Map: maintain collection of (key, value) pairs
  - Insert (k, v) : insert key k with associated value v
  - Remove (k) : remove record with key k
  - Find (k) : is key k present
  - A[k] (assuming map called “A”) : value associated with k

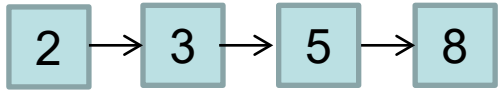
Set / Map Data Structures

Insert	Remove	Find
O(n)	O(n)	O(log n)
O(1)	O(1), post-find	O(n)
O(1), post-find	O(1), post-find	O(n)
O(1)	O(1), post-find	O(n)
“O(1)”	“O(1)”	“O(1)”
O(log n)	O(log n)	O(log n)

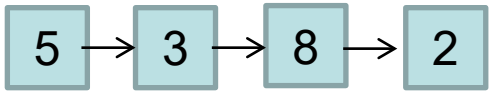
Sorted array

Unsorted array

Sorted (doubly) linked list



Unsorted (doubly) linked list



Hash table (we’ll study these soon…)

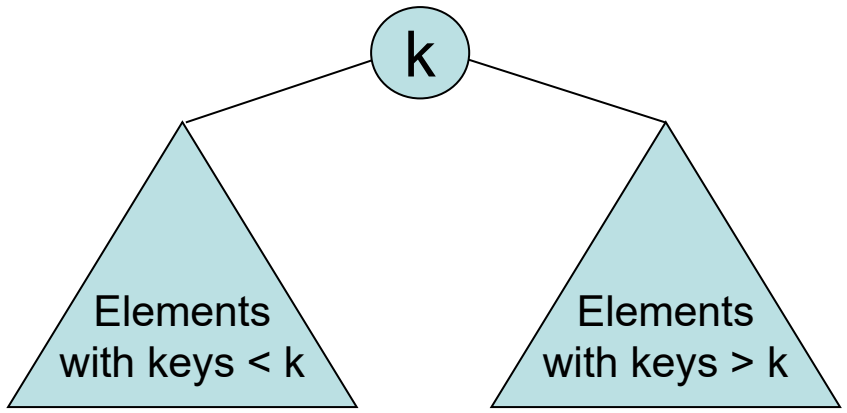
(Balanced) binary search trees, skip lists, and B-trees

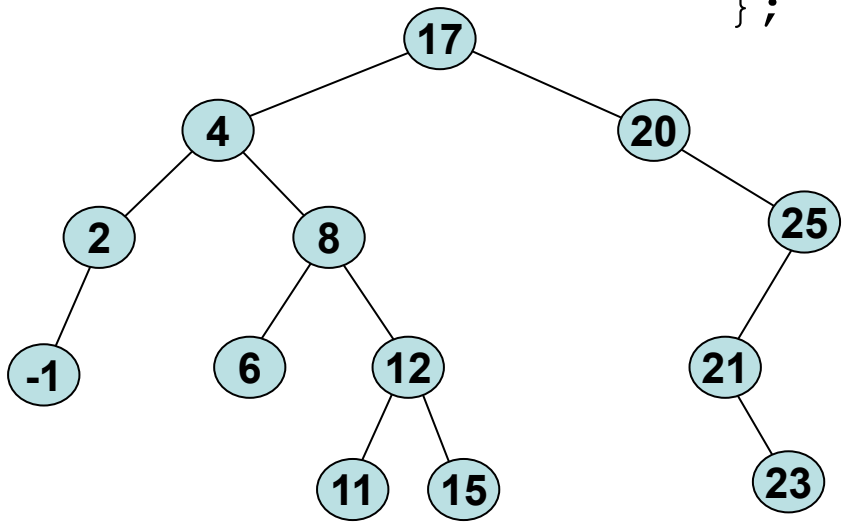
Although slightly slower than hash tables, BSTs and their relatives are comparison-based, and support many operations beyond insert, remove, and find that hash tables can’t handle.

Binary Search Trees

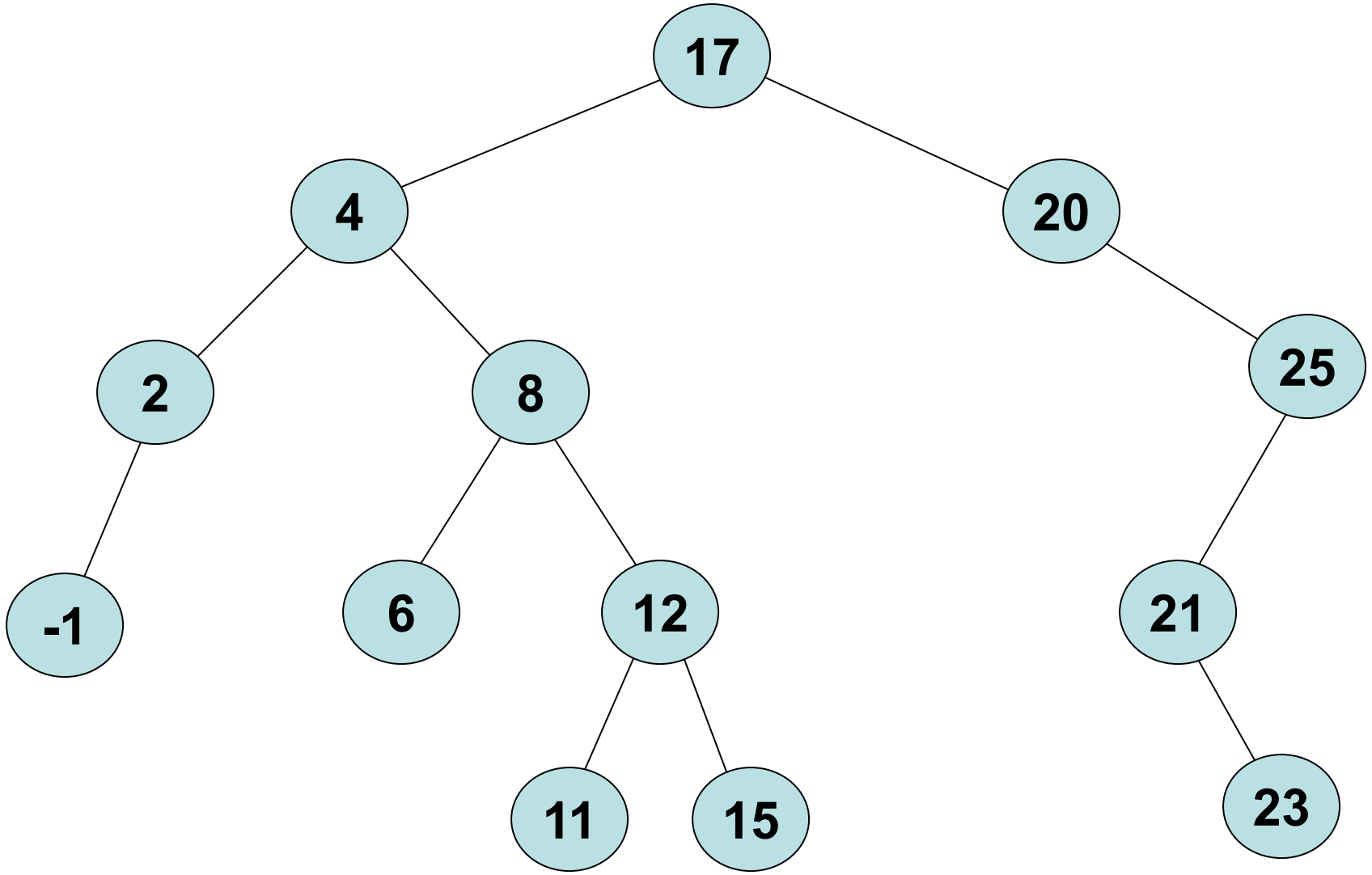
- A binary tree satisfying the **binary search tree property**.
- Each node typically points to its left child, right child and (sometimes) parent.
- **Balanced:** height = O(log n)

```
struct Node {
    int key;
    Node *left;
    Node *right;
    // Maybe:
    Node *parent;
};
```





The BST Is a "Horizontally-Ordered" Structure…

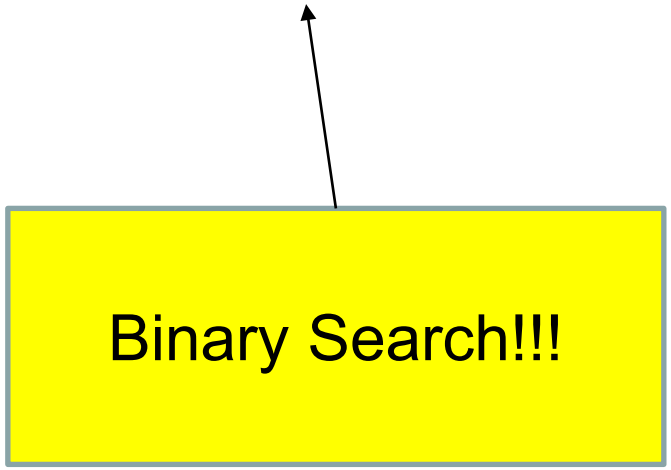


In contrast to heaps, which are "vertically" ordered.

Fundamental Operations

Most BST operations can be implemented in a simple recursive fashion in O(height) time (O(log n) if balanced).

```
Node *find(Node *T, int k) {
  if (T == NULL) return NULL;
  if (k == T->key) return T;
  if (k < T->key) return find(T->left, k);
  else return find(T->right, k);
}
```




```
struct Node {
   int key;
   Node *left;
   Node *right;
};
```

Fundamental Operations

Most BST operations can be implemented in a simple recursive fashion in O(height) time (O(log n) if balanced).

```
Node *find(Node *T, int k) {
  if (T == NULL) return NULL;
  if (k == T->key) return T;
  if (k < T->key) return find(T->left, k);
  else return find(T->right, k);
}
Node *insert(Node *T, int k) {
  if (T == NULL) return new Node(k);
  if (k < T->key) T->left = insert(T->left, k);
  else T->right = insert(T->right, k);
  return T;
}
```

```
struct Node {
   int key;
   Node *left;
   Node *right;
};
```

Balance Matters!

- What is the total running time for inserting the elements 1, 2, 3, …, N?
- What if we insert N random elements?
- Interesting fact: If we insert N random elements (or equivalently, insert any N elements but in random order), we get a tree that is balanced (i.e., having height O(log N)) with high probability.
  - Preview: we'll exploit this interesting fact to keep our trees balanced – by making sure they stay in a state that is "as if just built randomly from scratch".

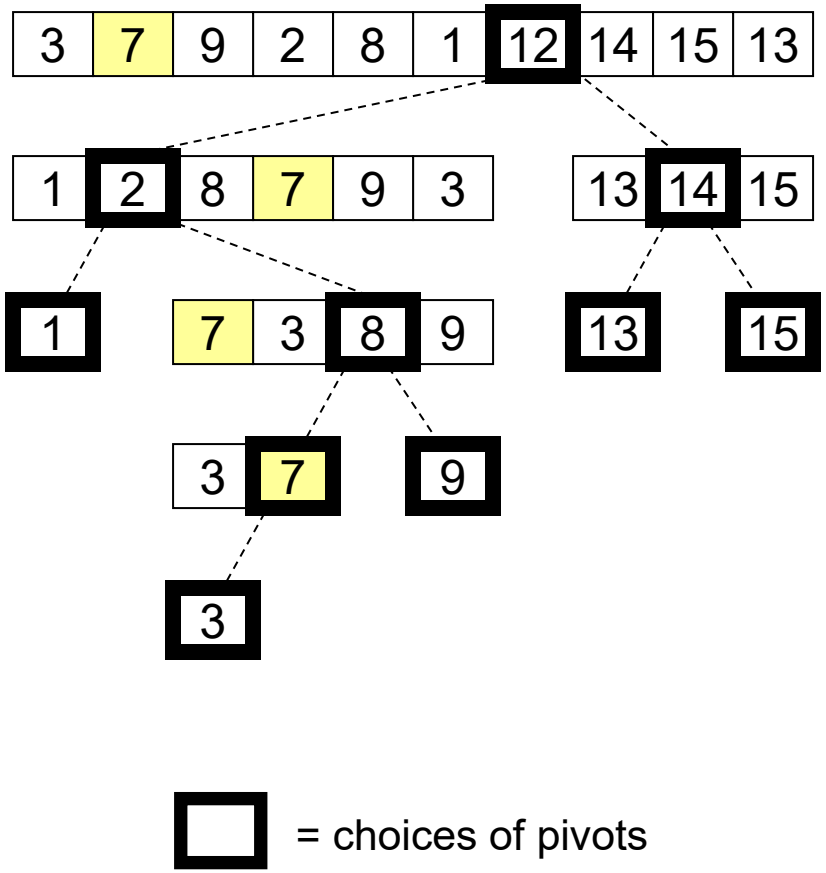
Quicksort and BST Construction: Essentially the Same!

- BST == quicksort recursion tree
- Quicksort and BST construction make same comparisons, just in a different order.
- Randomized quicksort analogous to building a BST by inserting elements in random order.

Randomly-built BST balanced whp.

⇕

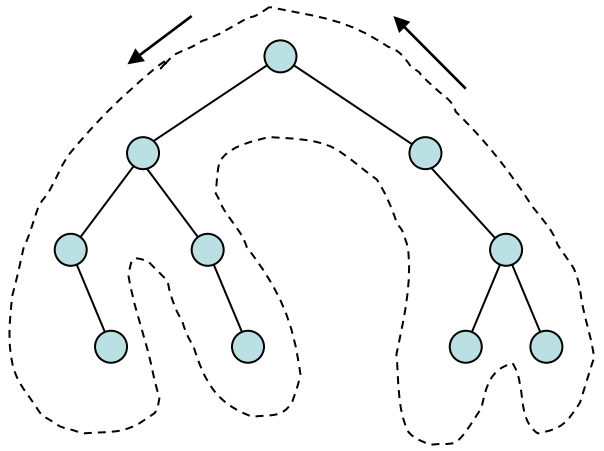
On every element, quicksort spends O(log n) comparisons whp.



Tree Traversals

- We often enumerate the contents of an n-element BST in O(n) time using an **inorder** tree traversal:

```
Inorder(T):
if T == NULL, then return
Inorder(T->left)
print T->key
Inorder(T->right)
```



- Other types of common traversals (for non-binary also):
  - Preorder: root, left subtree, right subtree.
  - Postorder: left subtree, right subtree, root.
  - Eulerian: root, left subtree, root, right subtree, root.

  (sequence of nodes encountered when we “walk around” a BST)

Sorting with a BST

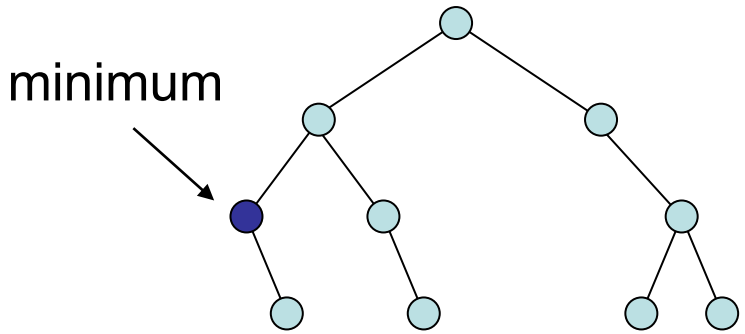
- An in-order traversal prints the n elements in a BST **in sorted order** in only O(n) time.
- Therefore, we can use BSTs to sort:
  - *insert* n elements O(n log n). (*)
  - then do an inorder traversal. O(n).

(*) if tree is kept balanced

- Recall: BSTs are comparison-based

Minimum and Maximum

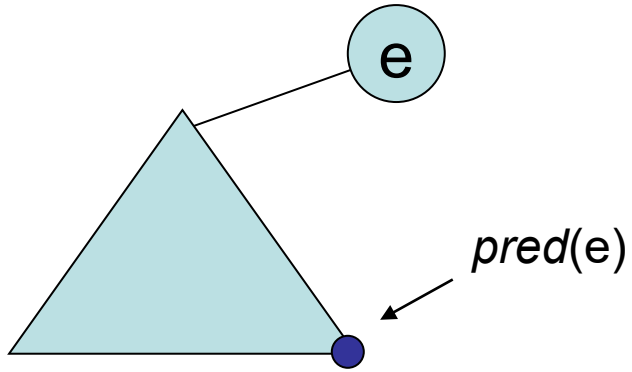
- It's easy to locate the minimum and maximum elements in a BST in O(h) time.
- For the minimum, start at the root and walk left as long as possible:



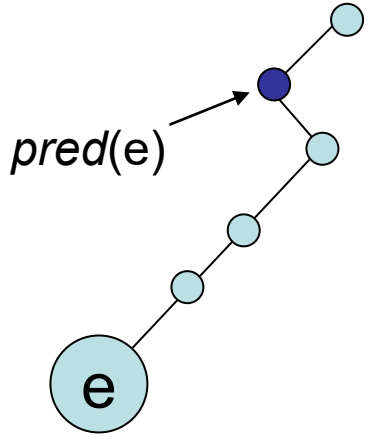
- Vice-versa for the maximum…

Predecessor and Successor

- The *pred*(e) operation takes a pointer to e and locates the element immediately preceding *e* in an inorder traversal.



If e has a left child, then *pred*(e) is the maximum element in *e's* left subtree.

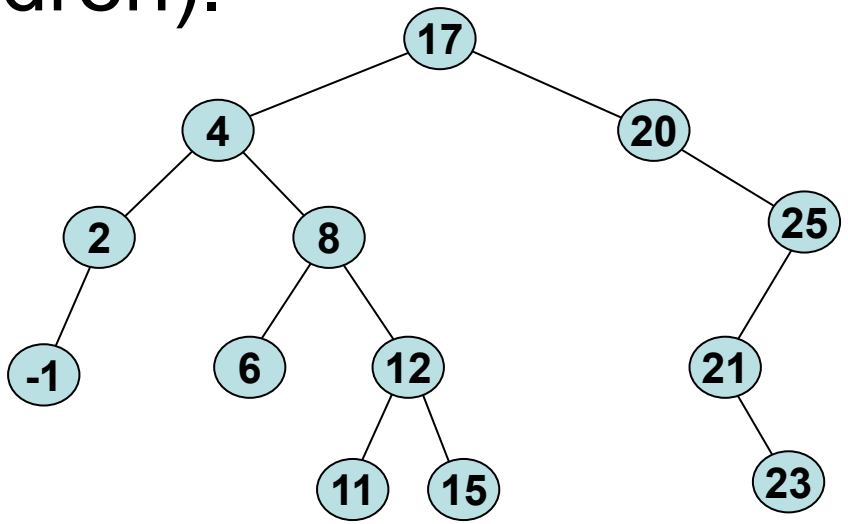


If e has no left child, then *pred*(e) is the first "left parent" we encounter when walking from e up to the root.

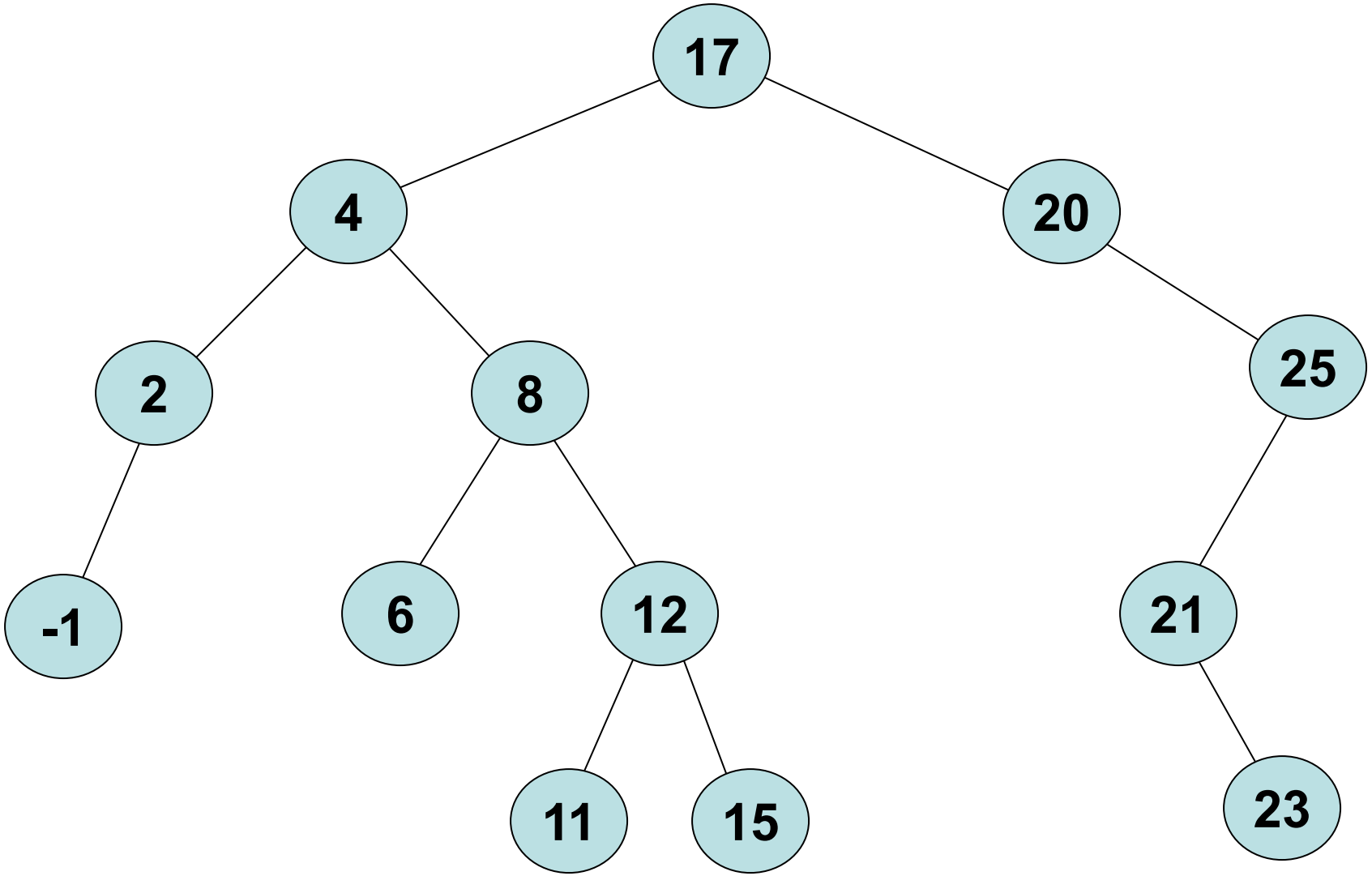
- The successor operation, succ(e), is completely symmetric to this. Both take O(height) time.

Deletion – a Standard but "Ugly" Approach…

- It's easy to *delete* a leaf node (no children).
- It's easy to *delete* a node with only one child – just replace with the child.
- To *delete* an element e with two children, first swap e with either *pred*(e) or *succ*(e).
  - If e has two children, *pred*(e) and *succ*(e) each will have at most one child.
  - Replacing e with *pred*(e) or *succ*(e) is o.k. with BST property.



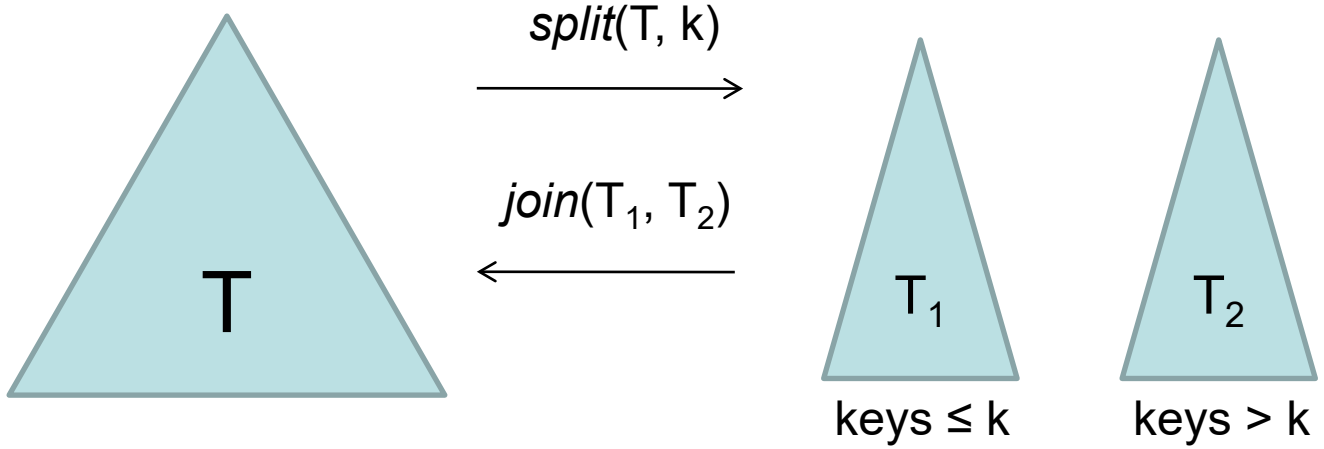
What Happens if you Search for 14?



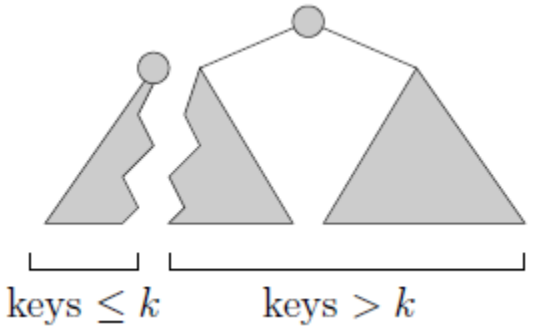
Inexact Searches and Range Queries

- In addition to *pred*(e) and *succ*(e) that take pointers to elements, we could implement:
  - *pred*(k) : find the element with largest key ≤ k.
  - *succ*(k) : find the element with smallest key ≥ k.
- This allows us to perform inexact searches: if an element with key k isn't present, we can explore nearby elements in the BST.
- Can also answer **range queries**: output all elements whose keys are in the range [a, b].

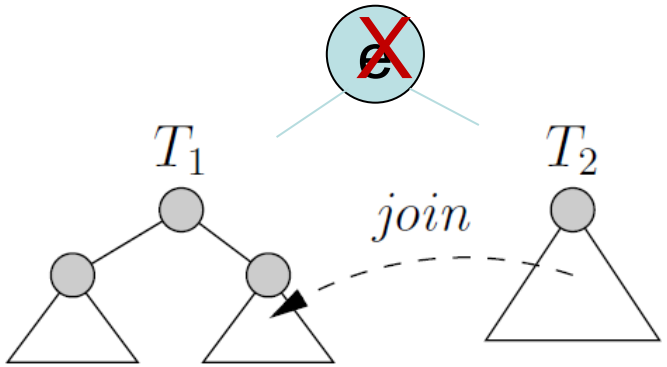
Split and Join



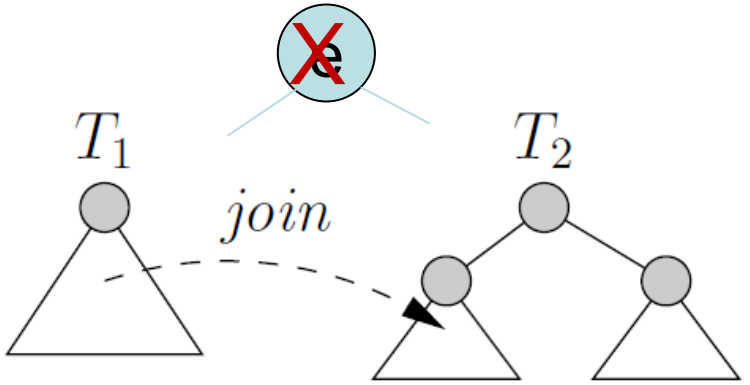
Recursive structure of *split*(T, k):



Simpler way to delete e: replace with join of children:



OR



T1->right = join(T1->right, T2);

T2->left= join(T1, T2->left);